

Zadatak

Implementirati *Secure Vault* – sistem za sigurno upravljanje tajnama (lozinke, API ključevi, sertifikati itd.) zasnovan na *zero-knowledge* arhitekturi. Cilj sistema je omogućiti razvojnim timovima da čuvaju i dijele osjetljive podatke na siguran način, pri čemu server (*backend*) nikada ne smije imati uvid u dekriptovani sadržaj tajni (*zero-knowledge* princip). Sistem prepoznaje tri grupe korisnika: *Admin*, *Team Lead* i *Developer*.

Funkcionalni zahtjevi sistema su:

1. Upravljanje tajnama (*Vault* mehanizam): Korisnici mogu da kreiraju, čitaju, ažuriraju i brišu tajne. Enkripcija tajni se mora vršiti isključivo na strani klijenta (u veb čitaču), prije slanja na server. Server u bazu podataka upisuje samo enkriptovani *blob*. Na klijentskoj strani se, prilikom registracije korisnika, generiše *master password*. Iz te lozinke se pomoću KDF algoritma (npr. Argon2 ili PBKDF2) generiše ključ za enkripciju podataka. *Master* lozinka se nikada ne šalje na server.
2. Sigurno dijeljenje (*Secure sharing*): *Team Lead* može podijeliti tajnu sa članovima tima (*Developer*). Budući da server ne može dešifrovati tajnu, dijeljenje se mora realizovati korišćenjem asimetrične kriptografije. Svaki korisnik, prilikom registracije, generiše par ključeva proizvoljnog asimetričnog algoritma. Javni ključ se čuva na serveru. Čuvanje privatnog ključa realizovati na proizvoljan način, ali tako da se ne naruše osnovni principi asimetrične kriptografije i *zero-knowledge* principa. Primjer dijeljenja tajne može da izgleda ovako: kada korisnik A dijeli tajnu sa korisnikom B, aplikacija na klijentskoj strani dešifruje tajnu ključem korisnika A, a zatim je ponovo enkriptuje javnim ključem korisnika B i šalje na server. Implementirati mogućnost opoziva pristupa dijeljenoj tajni (re-enkripcija tajne za preostale korisnike sa kojima je tajna podijeljena). Omogućiti vremenski ograničeno dijeljenje (tajna dostupna samo do definisanog roka). Evidentirati sve pokušaje pristupa dijeljenim tajnama u *audit* logu.
3. Organizacija: *Admin* upravlja korisničkim nalogima i definiše sigurnosne politike (npr. minimalna dužina *master* lozinke, obavezna rotacija tajni svakih n dana, trajanje sesije itd.).

Arhitektura sistema se sastoji od SPA klijenta i REST/GraphQL API servera. Osnovni sigurnosni zahtjevi obuhvataju:

1. Autentikacija i Autorizacija: Obavezna je implementacija više-faktorske autentikacije (MFA). U prvom koraku korisnik unosi korisničko ime i lozinku, a u drugom kôd dobijen pomoću TOTP (*Time-based One-Time Password*) algoritma i odgovarajuće *authenticator* aplikacije (npr. *Google Authenticator*). Dodatno, implementirati OIDC protokol za prijavu uz pomoć nekog eksternog naloga. Potrebno je implementirati zaštitu sesije korišćenjem *HttpOnly*, *Secure* i *Strict* flegova kolačića, uz rotaciju sesijskih tokena (trajanje *access* tokena treba da bude konfigurabilno, za potrebe testiranja rotacije). Potrebno je implementirati kombinovanu kontrolu pristupa zasnovanu na ulogama (RBAC) i atributima resursa (ABAC). Pristup tajnama zavisi od uloge korisnika, vlasništva nad tajnom, pripadnosti timu, roka važenja dijeljenja i

statusa opoziva. Dodatno, svaka sesija je vezana za karakteristike uređaja (*device fingerprinting*), te se pristup dozvoljava isključivo sa registrovanog uređaja.

2. *API Gateway & Rate Limiting*: sva komunikacija ka serveru prolazi kroz *API Gateway*. Implementirati napredni *rate limiting* (npr. pomoću *Redis*-a) kako bi se spriječili *brute-force* napadi na login *endpoint*. *API Gateway* treba da detektuje i blokira sumnjive IP adrese (npr. previše neuspješnih pokušaja dekripcije, sumnjive sekvence korisničkog unosa u zahtjevima, pokušaj korištenja isteklih sesijskih tokena itd.).
3. *Honeypot* sistem (detekcija upada): u bazi podataka moraju postojati „lažne tajne“ (*honeytokens*) koje nisu vidljive regularnim korisnicima kroz interfejs. Ukoliko neko pokuša da pristupi ovim zapisima (npr. putem *SQL Injection* napada, kompromitovanog admin naloga i sl.), sistem mora automatski da „zamrzne“ nalog aktivnog korisnika i pošalje upozorenje administratoru. Za potrebe testiranja validnosti ovog koncepta, administrator sistema bi trebalo da može privremeno „uključiti“ specijalno kreirani *endpoint* koji je ranjiv na *SQL Injection* napad. Omogućiti automatsko obavješćavanje putem e-maila i voditi statistiku pokušaja pristupa lažnim tajnama.
4. *Immutable Audit Log*: svaka akcija (pristup tajni, promjena, brisanje i sl.) se bilježi. Kako bi se osigurao integritet logova (da niko, pa ni *Admin* ne može obrisati tragove), svaki log zapis mora sadržati kriptografski heš kompletnog zapisa (koncept *blockchain*-a ili *Merkle tree*), čime se formira lanac povjerenja. Potrebno je konfigurirati Nagios za monitoring svih ključnih servisa u sistemu.

Tehnička realizacija podrazumijeva korištenje modernih *web*-baziranih tehnologija i *framework*-a za razvoj pojedinih dijelova sistema. Za implementaciju kriptografskih mehanizama, koristiti standardne biblioteke i alate, pri čemu nije dozvoljeno pisati sopstvene implementacije kriptografskih algoritama. Poželjno je da rješenje bude „kontejnerizovano“, tj. da se koristi *Docker* ili neki sličan alat radi bolje izolacije procesa i povećanja stepena sigurnosti sistema. Sve detalje zadatka koji nisu precizno specifikovani realizovati na proizvoljan način. Detalje korisničkog interfejsa realizovati na proizvoljan način.

Studenti koji uspješno odbrane projektni zadatak stiču pravo izlaska na usmeni dio ispita. Prije odbrane, potrebno je postaviti kompletan izvorni kod projektnog zadatka na *moodle*. Projektni zadatak važi od prvog termina januarsko-februarskog ispitnog roka 2026. godine i vrijedi do objavljivanja sljedećeg projektnog zadatka.